

Advanced C Programming Lab

Session X – May 28

Objectives

The topics we will be covering today are;

- C Function Pointers
- C Callback Functions

Function Pointers

A function pointer is like a normal pointer, but instead of pointing to a variable, it points to a function which means it stores the address of a function. This allows us to call that function using the pointer.

Function pointers let you decide which function to run while the program is running, or when you want to pass a function as an argument to another function.

A simple example would be a saved phone number; the pointer knows where the function lives in memory, so you can "call" it on demand.

Function Pointers

To declare a function pointer;

```
returnType (*pointerName)(parameterType1, parameterType2, ... );
```

Like: `int (*ptr)(int, int);`

This means `ptr` is a pointer to a function that takes two `int` values and returns an int.

Because a function pointer's name already represents its address in memory, a function can be assigned to a pointer in either ways below;

```
ptr = add;  == ptr = &add;
```

And to call an assigned function;

```
ptr(5, 3); == (*ptr)(5, 3);
```

Function Pointers

```c
#include <stdio.h>
int sum(int a, int b) {
  return a + b;
}
int main() {
  int (*ptr)(int, int) = sum;
  int result = ptr(7, 3);
  printf("Result: %d", result);
  return 0;
}
```

Output:

Result: 10

Declaring a function pointer just gives you a variable that can hold that address - so you can change it or pass it around.

Function Pointers

Function pointers can also be passed to other functions - which is called a **callback**.

This allows a function to call another function that you provide as input.

```
#include <stdio.h>
void greetMorning() { printf("Good morning!\n"); }
void greetEvening() { printf("Good evening!\n"); }
void greet(void (*func)()) {
  func();
}
int main() {
  greet(greetMorning);
  greet(greetEvening);
  return 0;
}
```

Output:

```
Good morning!
Good evening!
```

In this case out `greet()` takes another function as a parameter and calls it. This is commonly used in event-driven programs or libraries that need user-provided callback functions.

Function Pointers

We can also store several function pointers inside an array, so that we can choose which function to run while the program is running.

This is often used for simple menus, command lists, or calculators - anywhere you want to call different functions based on user input.

A great example of this would be a simple calculator that is provided with the inputs, but the user decides the action that program will run.

Function Pointers

```c
#include <stdio.h>
void sum(int a, int b) { printf("Result: %d\n", a + b); }
void sub(int a, int b) { printf("Result: %d\n", a - b); }
void multi(int a, int b) { printf("Result: %d\n", a * b); }
int main() {
  int choice, x = 6, y = 4;
  void (*operations[3])(int, int) = { sum, sub, multi };
  printf("x = %d, y = %d\n\n", x, y);
  printf("Choose an operation:\n");
  printf("0: Add\n1: Subtract\n2: Multiply\n");
  scanf("%d", &choice);
  if (choice >= 0 && choice < 3) {
    operations[choice](x, y);
  } else {
    printf("Invalid choice\n");
  }
  return 0;
}
```

Output:

x = 6, y = 4

Choose an operation:

0: Add

1: Subtract

2: Multiply

{?}

Callback Functions

A callback function is a function that is passed as an argument to another function.

The receiving function can then run it (call it back) whenever it needs to.

This is a powerful way to make your code flexible and reusable - you can decide which function should run, without changing the main logic.

Callback Functions

```c
#include <stdio.h>
void sayHi() {
  printf("Hi from callback!\n");
} // Normal Function
void runCallback(void (*callback)()) {
  printf("Before callback...\n");
  callback();
  printf("After callback.\n");
} // Receiver of a function pointer
int main() {
  runCallback(sayHi);
  return 0;
} // runs itself and calls sayHi
```

Output:

Before callback...
Hi from callback!
After callback.

Callback Functions

Passing functions that take parameters is also possible - just make sure the function pointer type matches.

```c
#include <stdio.h>
void addNum(int a, int b) {
  printf("Sum is: %d\n", a + b);
}
void calcu(void (*callback)(int, int), int x, int y) {
  callback(x, y);
}
int main() {
  calcu(addNum, 10, 15);
  return 0;
}
```

Output:

Sum is: 25

Thank you for your time and attention!

References:

- https://www.w3schools.com/
- https://www.geeksforgeeks.org/